



Project – Airline Reservation System

Object Oriented Programming

(CSEG1044)

Phase 1: Requirement and Use Cases

Members:-

Yash Jangir – 590014858

Parth Choudhary-590016913

Dushyant Singh-590012592

Soham Sharma-590017041

Anuj Pandey-590017258

1. Phase Goal and Focus

The primary goal of this phase is to convert the project idea into concrete use cases and measurable acceptance criteria. We must focus first on the workflow that best shows managing flights, passenger bookings, confirming bookings, and seat scheduling. The rest of the project should support that main workflow.

2. Detailed Actor Definitions

Before writing use cases, we need to establish who is using the system to ensure the actors are clearly identified:

- **Admin (Airline Manager / Flight Coordinator):** Responsible for managing flight routes, reviewing passenger bookings, and scheduling seat allocations.
- **Passenger:** Responsible for viewing available flights and booking tickets.

3. The Use Case Diagram Requirements

We are required to produce one use case diagram. Since we cannot submit vague use cases like "manage system", our diagram should include the specific action-oriented main use cases:

- **Admin actions:** Create a flight, Confirm passenger bookings, Assign seat allocations, View flight-wise status (Passenger Manifest).
- **Passenger actions:** Record passenger booking.

(Note: Set 1 of your group should draft this diagram using a tool like Lucidchart, Draw.io, or PlantUML).

4. Fully Written Use Cases

We must provide at least fully written use cases that include both the success path and the failure path for each. It's crucial to describe user goals rather than just describing screens.

Use Case 1: Create a Flight

- **Actor:** Admin
- **Goal:** To open a new flight schedule for passengers to book.
- **Success Path:**
 - The Admin selects a registered Route from the system.
 - The Admin enters flight details (airplane model, capacity, ticket price, departure date/time).
 - The Admin submits the form.
 - The system validates the data and saves the new flight to the database.
 - The system displays a success message, and the flight becomes visible to passengers.
- **Failure Path (Validation/Conflict):**
 - If the Admin leaves required fields empty or enters a past departure date, the system rejects the submission and displays an error message.

Use Case 2: Record Passenger Booking

- **Actor:** Passenger
- **Goal:** To book a ticket for an active flight.
- **Success Path:**
 - The Passenger selects an active flight from their dashboard.
 - The Passenger clicks the "Book" button.
 - The system checks flight availability (e.g., remaining capacity match).

- The system records the booking and updates the status to "PENDING".
- The Passenger sees a confirmation message.
- **Failure Path (Validation/Conflict):**
 - If the Passenger has already booked this specific flight, the system detects a duplicate and rejects the booking, notifying the passenger.
 - If the flight has reached maximum capacity, the system prevents the booking and explains why.

Use Case 3: Confirm Bookings

- **Actor:** Admin
- **Precondition:** Bookings exist for a specific flight.
- **Goal:** To filter pending bookings and mark selected passengers as confirmed for the flight.
- **Success Path:** The Admin pulls up the booking list for a flight, selects a passenger, and updates their status to "CONFIRMED". The system commits this change via the DAO layer and updates the UI.
- **Failure Path:** The Admin attempts to confirm a passenger who has already cancelled their booking. The system prevents the state change and shows an error indicating invalid workflow progression.

Use Case 4: Assign Seat Allocations

- **Actor:** Admin
- **Goal:** To schedule a specific seat number and class for a confirmed passenger.
- **Success Path:**
 - The Admin selects a flight and views the list of "CONFIRMED" passengers.
 - The Admin selects a specific passenger.
 - The Admin enters a seat number and class type (e.g., Economy, Business).

- The system saves the seat allocation.
- The Passenger's dashboard updates to show the assigned seat.
- **Failure Path (Validation/Conflict):**
 - If the chosen seat overlaps with another seat already assigned to someone else, the system blocks the assignment and prompts the Admin to choose a new seat.
 - If the Admin tries to assign a seat to a passenger whose status is not "CONFIRMED", the system denies the action.

Use Case 5: View Flight-Wise Status

- **Actor:** Admin or Passenger (Different views based on role)
- **Goal:** To see the progress and metrics of a flight.
- **Success Path:** The user selects a flight to view its status. For the Admin, the system generates flight-wise booking reports showing total capacity, confirmed bookings, and assigned seats. For the Passenger, the system displays only their personal booking status (e.g., "Pending", "Confirmed", "Seat Assigned").

5. Acceptance Criteria for the Most Important Workflow

Our guidelines recommend focusing on the workflow that best shows managing flights, passenger bookings, confirmations, and seat scheduling. Here are the measurable acceptance criteria for this core end-to-end workflow:

- **Criteria 1 (Flight Creation):** The system must allow the Admin to save a new flight only if all mandatory fields (Route, Model, Capacity, Date) are filled out with valid data.
- **Criteria 2 (Booking Validation):** A passenger must be able to book an active flight, but the system must strictly prevent them from submitting more than one booking to the same flight.
- **Criteria 3 (Confirmation):** The system must allow the Admin to view all bookings for a specific flight and successfully change a passenger's status from "PENDING" to "CONFIRMED".
- **Criteria 4 (Scheduling Verification):** The system must successfully save a seat allocation for a confirmed passenger and immediately make that seat visible on the passenger's view. It must successfully reject seat assignment if a seat conflict exists.